

Documento B — Processo de Teste

Projeto: Finansme Flutter

Tecnologia: Flutter / Dart

Norma aplicada: ISO/IEC/IEEE 29119-2

1. Estratégia de Teste

- Testes de unidade da camada ApiClient e dos serviços de domínio
- Testes de integração de API com servidor HTTP mock (TestApiServer)
- Validação de contratos de requisição e resposta JSON
- Validação de autenticação via header Authorization (Bearer Token)
- Validação das regras de negócio de saldo, movimentações e cadastros
- Cobertura de cenários positivos e negativos (status 200/201, 400, 401, 403, 404, 409, 422)

2. Ambiente de Teste

- Flutter SDK
- Dart SDK
- Pacote flutter_test
- Pacote integration_test
- TestApiServer (servidor HTTP local para mocks)
- ApiClient (pacote finansme_flutter/src/services/api_client.dart)
- Sistema operacional: macOS / Linux / Windows

3. Critérios de Entrada

- Projeto Flutter compilando sem erros
- ApiClient e serviços de domínio implementados
- TestApiServer implementado em test/support/test_api_server.dart
- Endpoints REST do back-end especificados e versionados
- Documento A — Base Conceitual de Teste concluído e aprovado

4. Critérios de Saída

- 100% dos casos de teste planejados executados
- 0 reprovações em casos de teste críticos (autenticação, saldo, cadastros)
- Defeitos identificados registrados e tratados
- Resultado de cada execução salvo em log
- Relatório consolidado de execução produzido

5. Ordem de Execução

1. Executar testes do módulo User (signin → login → update)
2. Executar testes do módulo Account (create → list → balance)
3. Executar testes do módulo Account Plan
4. Executar testes do módulo Transactor
5. Executar testes do módulo Financial Movement (create → list → update → delete)
6. Executar testes do módulo AI Advisor
7. Executar testes de integração que envolvem saldo (receita + despesa)

6. Implementação

Estrutura de diretórios de teste:

```
test/  
support/  
test_api_server.dart  
user_test/  
signin_test.dart  
login_test.dart  
update_user_test.dart  
account_test/  
create_account_test.dart  
list_accounts_test.dart  
balance_test.dart  
account_plan_test/  
account_plan_test.dart  
transactor_test/  
transactor_test.dart  
financial_movement_test/  
create_movement_test.dart  
list_movements_test.dart  
update_movement_test.dart  
delete_movement_test.dart  
ai_advisor_test/  
ai_advisor_test.dart
```

7. Controle

- Planejados — todos os CTs descritos no Documento C
- Executados — quantidade total de CTs rodados na suíte
- Aprovados — CTs cujo resultado obtido = resultado esperado
- Reprovados — CTs cujo resultado obtido != resultado esperado
- Bloqueados — CTs não executáveis por dependência (ex.: ambiente)
- Cobertura — % de requisitos RF cobertos por CTs aprovados

8. Execução

Comandos utilizados para execução da suíte de testes:

```
flutter test  
flutter test test/user_test  
flutter test test/account_test  
flutter test test/account_plan_test
```

```
flutter test test/transactor_test  
flutter test test/financial_movement_test  
flutter test test/ai_advisor_test  
flutter test --coverage
```

9. Conclusão

O processo será encerrado após a execução completa da suíte automatizada, registro dos resultados de cada CT, análise dos defeitos identificados e geração do relatório consolidado. Os critérios de saída devem ser atendidos antes da finalização do ciclo de teste.